VERIQTA

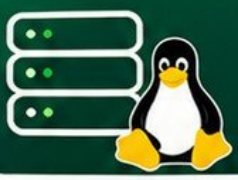
50 Linux Interview Scenarios

Real Production Problems, Root Causes, Fixes and Engineering Decisions



- Performance & Processes
- Memory & OOM
- Storage & Filesystems
- Networking & DNS
- Security & Access
- Containers & systemd
- Deployment & Automation
- Monitoring & Logging
- Cloud & Reliability
- Real-World Projects

@veriqta Instagram · @veriqta GitHub · @veriqta LinkedIn · @veriqta Telegram · @veriqta X · @veriqta Discord



The Server Has Spare CPU, but Production Is Unusable

Scenario 01

CPU utilization is 25%, but load average exceeds 80.

What to Investigate

- Tasks blocked in uninterruptible sleep (D state).
- I/O wait and storage latency.
- Run queues and process scheduling.
- Blocked kernel operations (e.g., disk, network, or NFS).

Key Concepts

- Load average = runnable + uninterruptible (I/O) tasks.
- CPU utilization = actual CPU time used.
- High load can occur due to I/O, locks, or resource contention, not high CPU.

Useful Commands

- uptime / w
- top / htop
- vmstat 1
- iostat -x 1
- ps -eo stat,comm,pid | grep D

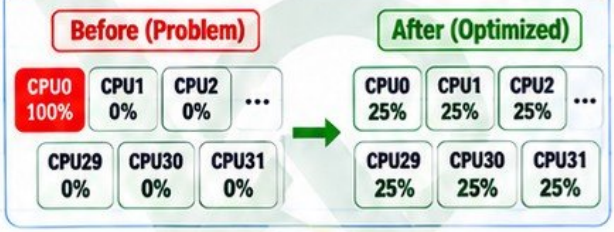
Scenario 02

One CPU core is saturated while the other 31 are idle.

What to Investigate

- Single-threaded execution (e.g., a process using one core).
- CPU affinity and core binding (taskset, numactl).
- IRQ placement and interrupt load.
- Scheduler behavior and application concurrency.

Illustration: Single-Core Bottleneck



Useful Commands

- top -H -p <pid>
- ps -L -p <pid>
- taskset -p <pid>
- mpstat -P ALL 1
- grep . /proc/interrupts

Scenario 03

Application latency doubles after moving to a larger virtual machine.

What to Investigate

- CPU steal time (hypervisor contention).
- NUMA locality (memory access across nodes).
- Virtualization effects (vCPU scheduling, overcommit).
- CPU frequency scaling (performance vs. power saving).
- Thread scheduling and concurrency contention.

Key Checks and Evidence

Check	Command
CPU steal time	vmstat 1 | awk '{print $15}'
NUMA topology	numactl --hardware
CPU frequency	cpufreq-info | grep "current"
vCPU scheduling	mpstat -P ALL 1
Application latency	curl -w "%{time_total}\n" <URL>

What to Look For

- High %steal indicates hypervisor contention.
- Remote NUMA memory access increases latency.
- Lower CPU frequency than expected.
- Increased context switching or run queue length.

Production Project

Build a Linux performance incident lab that reproduces three different bottlenecks with similar user-facing symptoms. Produce a perf, pidstat, vmstat, and mpstat evidence report.

1. Set Up Test Environment (VM or EC2)
2. Reproduce 3 Bottlenecks (CPU, I/O, Scheduling)
3. Collect Evidence (perf, pidstat, vmstat, mpstat)
4. Analyze and Identify Root Causes
5. Produce a Findings and Remediation Report



The Memory Leak That Wasn't a Memory Leak

Scenario 04

Linux reports 95% memory usage, but applications are healthy.

What to Investigate

- Break down memory: used, free, buffers, and cache.
- Identify reclaimable memory (page cache and slabs).
- Check for genuine memory pressure (swap, OOM risk).
- Verify application memory (RSS) and overall system health.

Memory Breakdown (Example)

Item	Meaning	Action
Page Cache	File data in RAM (reclaimable)	Can be freed when needed
Buffers	Kernel I/O buffers (reclaimable)	Usually safe
RSS (Resident)	Actual app memory (in use)	Monitor for leaks
Swap	Memorymoved to disk (pressure indicator)	Investigate if increasing

Useful Commands

- free -h
- vmstat 1
- cat /proc/meminfo
- smem -r
- top / htop
- sar -r 1

Scenario 05

A container is OOM-killed while the host has gigabytes of available RAM.

What to Investigate

- Check cgroup memory limits (--memory, memory.max).
- Examine anonymous memory (app allocations) vs. page cache.
- Review memory.events for OOM triggers.
- Evaluate oom_score_adjj and which process was killed.

Container Memory and OOM Flow

Host (Available RAM)
Total: 16 GB
Used: 6 GB
Free: 10 GB

→

Container (cgroup Limit)
Limit: 2 GB
Used: 2 GB
OOM Triggered

→

Result
OOM-Killed
(Exit Code 137)

Useful Commands

- docker stats
- cat /sys/fs/cgroup/memory.max
- cat /sys/fs/cgroup/memory.events
- ps -eo pid,oom_score_adj,cmd
- dmesg | grep -i oom

Scenario 06

A Java application becomes slower after increasing itts heap allocation.

What to Investigate

- Analyze GC behavior (frequency, pause time, and heap usage).
- Check for memory pressure (swapping and system reclaim).
- Assess impact on other workloads (less RAM for OS and cache).
- Review GC logs and JVM metrics (Old Gen, Full GC, and allocation rate).

Heap Size vs. Performance

Before (Heap: 4 GB)
- GC: ~0.5 s
- Throughput: High
- Latency: ~200 ms

→

After (Heap: 12 GB)
- GC (Full): ~5.2 s
- Throughput: Lower
- Latency: ~400 ms

Useful Commands

- jstat -gc <pid> 1s
- jcmd <pid> GC.heap_info
- tail -f gc.log
- vmstat 1
- free -h
- top -p <pid> / htop

Production Project

Construct a memory-forensics laboratory with host-level and cgroup-level pressure. Produce an OOM timeline and justify a remediation without blindly increasing RAM.

1. Set Up Lab Environment (Host + Containers)
2. Generate Memory Pressure (Controlled Workloads)
3. Capture Evidence (free, vmstat, memory.events, GC logs, dmesg)
4. Build OOM Timeline and Root Cause
5. Recommend and Test a Remediation

The Linux Process That Refuses to Die

Scenario 07 — A process remains after kill -9.

What to Investigate

- Determine if it's a zombie (defunct) or a task in uninterruptible sleep (D state).
- Identify the parent process (PPID).
- Check for blocked kernel I/O (e.g., disk, NFS, or device).
- Review process state, wait channel, and stack trace.

Process States and Meaning

State	Meaning	Common Cause
Z	Zombie (defunct)	Process exited; parent hasn't reaped it (wait).
D	Uninterruptible sleep	Blocked on I/O (e.g., disk, NFS, device).
R	Running	Actively using CPU.
S	Sleeping	Waiting for an event (e.g., I/O, timer).

Useful Commands

- ps -eo pid,ppid,state,cmd
- top / htop
- pstree -p <pid>
- cat /proc/<pid>/stack
- dmesg | tail
- lsof -p <pid>

Scenario 08 — A service repeatedly restarts even though its code exits successfully.

What to Investigate

- Check systemd unit configuration (Type, Restart, RestartSec).
- Review exit status and signals (systemctl status).
- Verify watchdog settings (WatchdogSec, sd_notify).
- Check service dependencies and start order (After, Requires).

Systemd Restart Behavior

Directive	Example	Effect
Restart	Restart=always	Restart regardless of exit status.
RestartSec	RestartSec=5s	Wait 5 seconds before restart.
Type	Type=simple	Main process = service.
WatchdogSec	WatchdogSec=30s	Restart if no health signal in 30s.

Useful Commands

- systemctl status <svc>
- systemctl cat <svc>
- journalctl -u <svc> -f
- systemctl show <svc> \ -p Restart,Type,ExitCode
- systemd-analyze critical-chain

Scenario 09 — A production application stops accepting connections without crashing.

What to Investigate

- Check file descriptor usage (lsof, /proc/<pid>/fd).
- Validate the open file limit (RLIMIT_NOFILE / ulimit).
- Look for socket leaks (ESTABLISHED/TIME_WAIT).
- Confirm EMFILE ("Too many open files") errors in logs.

File Descriptor Exhaustion Flow



Key Indicator: EMFILE = Too many open files

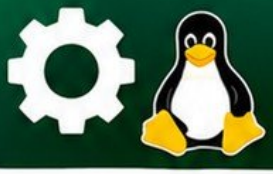
Useful Commands

- lsof -p <pid> | wc -l
- cat /proc/<pid>/limits
- ulimit -n
- ss -s
- ss -tanp | head
- grep "EMFILE" /var/log/*

Production Project

Bui d a process-failure simulator that generates zombie processes, restart loops, and file descriptor exhaustion. Create a recovery runbook that preserves evidence.

1. Set Up Lab (VM or Container)
2. Simulate Failures (Zombie, Restart Loop, FD Exhaustion)
3. Collect Evidence (ps, systemctl, lsof, journalctl)
4. Build Recovery Runbook (Safe and Reproducible)
5. Validate Fix and Prevent Recurrence



The Kernel Is Doing Work Nobody Requested

Scenario 10

System CPU usage spikes while application CPU remains low.

What to Investigate

- High kernel CPU time ('sy' in top/vmstat).
- Interrupts (IRQ) and softirqs (e.g., network or disk).
- Context switch rate and scheduler activity.
- Network processing (packet flood, high PPS, or NIC driver).

Where the CPU Can

CPU Type	What It Means	Typical Causes
us	User space (app)	Application work
sy	Kernel space	I/O, network, driver, system calls
si	Software interrupt	Network packet handling (NAPI, ksoftirqd)
hi	Hardware interrupt	Device IRQs (NIC, disk)
cs	Context switches	High task churn or I/O wait

Useful Commands

- top / htop
- vmstat 1
- mpstat -P ALL 1
- cat /proc/interrupts
- cat /proc/softirqs
- sar -n DEV 1
- ethtool -S <iface>

Scenario 11

A server freezes for seconds without any obvious CPU spike.

What to Investigate

- Scheduler stalls and run queue blocking.
- Lock contention (e.g., filesystem or kernel locks).
- Memory reclaim (page scanning, swapping).
- Blocked tasks using perf, PSI, and kernel logs.

Investigation Tools and Signals

Tool / Metric	What It Shows	Why It Matters
perf top	Kernel hotspots and locks	Identify blocking functions
PSI (Pressure Stall Info)	CPU, memory, I/O pressure	Shows resource contention
dmesg	Kernel events and warnings	Detect stalls, I/O errors, or timeouts
vmstat	Run queue, swap, I/O wait	Confirm resource pressure
cat /proc/loocks	File/FS lock holders	Find lock contention

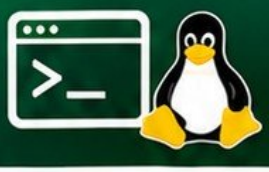
Useful Commands

- perf top
- cat /proc/pressure/*
- vmstat 1
- dmesg -T | tail -n 100
- cat /proc/locks
- pidstat -w /
- sysctl kernel.*stall*

Production Project

Create a kernel-behavior investigation lab using controlled load generation and performance counters. Compare user-space and kernel-space bottlenecks.

1. Set Up Lab (VM/EC2 + Workloads)
2. Generate Bottlenecks (CPU, I/O, Network)
3. Collect Evidence (perf, vmstat, mpstat, PSI)
4. Compare User vs. Kernel CPU Usage
5. Produce Findings and Remediation Plan



The Application Is Running, but systemd Says It Failed

Scenario 12

A service works manually but fails during boot.

What to Investigate

- Compare manual vs. systemd environment (PATH, variables).
- Check WorkingDirectory and file permissions (User, Group).
- Verify service dependencies (After, Requires, Wants).
- Ensure network availability (network-online.target).

Key systemd Checks

Check	Command	Purpose
Service status	systemctl status <svc>	See exact failure and logs.
Service unit	systemctl cat <svc>	Review configuration.
Boot logs	journalctl -u <svc> -b	Check failure during boot.
Dependencies	systemctl list-dependencies <svc>	Verify required units are up.

Useful Commands

- systemctl status <svc>
- journalctl -u <svc> -b
- systemctl cat <svc>
- systemctl show <svc>
- systemctl list-dependencies <svc>
- systemd-analyze blame

Scenario 13

A service passes health checks but silently stops processing jobs.

What to Investigate

- Validate watchdog design (WatchdogSec, sd_notify).
- Distinguish readiness vs. liveness checks.
- Monitor application progress metrics (job queue, throughput).
- Ensure failure detection and automatic recovery (Restart, RestartSec).

Health Check vs. Real Work

Check	Meaning	Risk if Misused
Liveness	Is the process running?	Service may be alive but doing no work.
Readiness	Is the service ready to handle work?	Traffic may be sent too early.
Watchdog	Service reports it is still healthy (e.g., sd_notify).	Without rerioidic updates, silent failures go undetected.
Progress Metric	Is work actually being processed (e.g., queue, rate)?	Health checks can be green while jobs are stuck.

Useful Commands

- systemctl show <svc> | grep Watchdog
- journalctl -u <svc>
- systemctl is-active <svc>
- systemctl restart <svc>
- watch -n 5 "queue_metric"
- systemctl status <svc> --no-pager

Production Project

Engineer a production-style systemd service with startup dependencies, resource limits, watchdog integration, structured logging, and controlled recovery testing.

systemd Service Lab

Service Design (systemd)

```
[Unit]
Description=Job Processor
After=network-online.target

[Service]
WorkingDirectory=/opt/app
ExecStart=/usr/bin/worker
User=app
Restart=on-failure
WatchdogSec=30s
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

Features to Implement

- Startup depndency (network-online)
- Resource limits (e.g., LimitNOFILE)
- Watchdog integration (sd_notify)
- Structured logging (journalctl)
- Auto-recovery (Restart policy)
- Failure and recovery testing

Deliverables

- Working service unit
- Logs and test results
- Failure and recovery report
- Recommendations



The Disk Has Free Space, but Linux Says It Is Full

Scenario 14

df -h reports 100% usage while du accounts for much less.

What to Investigate

- Check for deleted-but-open files (used space not released).
- Verify filesystem reservations (e.g., ext4 reserved blocks).
- Confirm correct mount points and filesystem boundaries.
- Identify large files in other mounted filesystems.

Key Checks and Evidence

Check	Command
Disk usage	df -hT
Actual usage	du -xsh /* | sort -h
Open deleted files	lsof | grep "(deleted)"
Reserved blocks	tune2fs -l <device> | grep -i "Reserved"
Mount points	findmnt -T <path>

Useful Commands

- lsof +L1
- df -hT
- du -xsh
- findmnt
- tune2fs -l <device>
- lsblk -f

Scenario 15

Applications cannot create files even though free disk space remains.

What to Investigate

- Check for inode exhaustion (no free inodes).
- Review filesystem quotas (user, group, or project).
- Identify directory entry growth (too many small files).
- Verify filesystem type and limits (e.g., ext4, xfs).

Disk Space vs. Inodes

Metric	Command	Healthy Example	Problem Example
Disk space	df -h	50% used	50% used
Inode usage	df -i	60% used	100% used
User quota	quota -u <user>	Within limit	Quota exceeded
Large directory	ls -l | wc -l	Few files	Millions of files

Useful Commands

- df -i
- quota -u <user>
- repquota -a
- find <dir> -type f | wc -l
- ls -l | wc -l
- xfs_quota -x -c "report"

Scenario 16

A filesystem becomes read-only during production traffic.

What to Investigate

- Check for filesystem errors (journal, metadata, I/O errors).
- Review kernel and storage logs (dmesg, journalctl).
- Investigate underlying storage (EBS, SAN, disk health, I/O timeouts).
- Plan a safe recovery (fsck, remount, or storage replacement).

Failure Investigation Flow

Check Logs (dmesg / journalctl) → Identify Filesystem Error (e.g., I/O, journal) → Safe Recovery (fsck / remount / replace disk) → Verify and Restore

Useful Commands

- dmesg | grep -i "ext4\|xfs\|I/O"
- journalctl -k -b
- mount | grep "ro,"
- fsck -n /dev/<device>
- fsck -y /dev/<device>
- mount -o remount,rw <mount>

Production Project

Build a filesystem failure laboratory that covers: open-deleted files, inode exhaustion, and read-only remounts. Restore service without destroying forensic evidence.

1. Set Up Lab (VM or EC2,
2. Reproduce Failures (open-delete, inode exhaustion, read-only)
3. Collect Evidence (df, du, lsof, dmesg, journalctl)
4. Recover Safely (fsck, remount, quota tuning)
5. Validate and Document (Root Cause + Fix)



The Storage Upgrade That Made the Database Slower

Scenario 17

Storage throughput improves, but database latency increases.

What to Investigate

- Compare IOPS, throughput (MB/s), and latency (ms).
- Check storage queue depth (eg., iostat -x, /sys/block/*/).
- Analyze database I/O pattern (random vs. sequential).
- Verify filesystem and mount options (e.g., noatime, discard).

Key Storage Metrics

Metric	Meaning	Check Command
IOPS	I/O operations per second	iostat -x 1
Throughput	Data transfer rate (MB/s)	iostat -x 1
Queue Depth	Pending I/O requests	iostat -x 1
Latency	Time per I/O (ms)	iostat -x 1

Useful Commands

- iostat -x 1
- sar -d 1
- pidstat -d -p <pid>
- blktrace / blkparse
- fio (I/O test)
- dstat -d

Scenario 18

A backup process causes unpredictable application outages.

What to Investigate

- Check for page-cache competition (app vs. backup).
- Monitor dirty-page writeback (vmstat, /proc/meminfo).
- Review I/O scheduling policy (e.g., mq-deadline, none).
- Identify and apply throttling (ionice, cgroups, rate limits).

I/O Contention Flow



Key Check: vmstat 1 | iostat -x 1 | pidstat -d

Useful Commands

- vmstat 1
- iostat -x 1
- ionice -c2 -n7 <cmd>
- systemd-run --scope \ --slice=backup.slice
- cat /proc/meminfo
- ps -eo pid,cmd,ionice

Scenario 19

A Linux VM fails to boot after a storage expansion.

What to Investigate

- Verify partition tables (lsblk, fdisk -l, parted -l).
- Check LVM configuration (pvs, vgs, lvs).
- Confirm filesystem resizing (resize2fs, xfs_growfs).
- Validate /etc/fstab and UUID changes (blkid, mount -a).

Boot Recovery Checklist

Step	Command	Purpose
1. Check disks	lsblk, fdisk -l	Confirm new disk/partition
2. Check LVM	pvs, vgs, lvs	Verify volumes and sizes
3. Resize FS	resize2fs or xfs_growfs	Extend filesystem
4. Check fstab	cat /etc/fstab	Validate UUID and mount
5. Test mount	mount -a	Confirm successful boot setup

Useful Commands

- lsblk -f
- fdisk -l / parted -l
- pvs | vgs | lvs
- resize2fs /dev/...
- xfs_growfs /mount
- blkid
- mount -a

Production Project

Build a multi-volume storage benchmark and failure-recovery lab. Demonstrate performance isolation, safe expansion, and rollback.





DNS Resolves Correctly, but the Application Still Cannot Connect

Scenario 20

dig succeeds while the application reports DNS failures.

What to Investigate

- Compare system vs. application DNS resolution (libc, Go, Java, etc.).
- Review /etc/nsswitch.conf (lookup order and sources).
- Check search domains and FQDN resolution.
- Inspect systemd-resolved (cache, stub resolver, and config).

DNS Resolution Paths

Component	What to Check
Resolver library	glibc, musl, JVM, Go resolver
nsswitch.conf	lookup order (files, dns, mdns)
Search domains	/etc/resolv.conf, domain suffixes
systemd-resolved	Cache, upstream DNS, stub (127.0.0.53)
App caching	Internal DNS cache and TTL

Useful Commands

- dig +short example.com
- getent hosts example.com
- cat /etc/nsswitch.conf
- resolvectl status
- resolvectl query example.com
- systemd-resolve --flush-caches

Scenario 21

DNS changes propagate, but some processes keep using old addresses.

What to Investigate

- Verify DNS TTL and caching behavior (system and app level).
- Check long-lived connections (e.g., HTTP, DB, message queues).
- Inspect connection pooling and resolver refresh settings.
- Confirm process restart or cache invalidation strategy.

Why Old Addresses Persist

Cause	Explanation
TTL (Time to Live)	Client caches DNS for the TTL duration.
Connection pooling	Existing connections stay open to old IP.
Long-lived sockets	Processes do not re-resolve DNS often.
App caching	Application maintains its own DNS cache.
No restart/refresh	Process not reloaded after DNS change.

Useful Commands

- dig example.com +ttl
- ss -tunap | grep <port>
- lsof -iTCP -sTCP:ESTABLISHED
- systemctl restart <service>
- grep -icache /etc/<app>/*

Scenario 22

A Linux host intermittently loses connectivity under high request volume.

What to Investigate

- Check conntrack table usage and limits (nf_conntrack).
- Investigate ephemeral port exhaustion (ip_local_port_range).
- Review socket states (ESTABLISHED, TIME_WAIT, SYN_SENT).
- Look for packet drops and resource saturation (netstat, ss, dmesg).

Key Networking Indicators

Metric / Setting	Command	What It Shows
Conntrack usage	conntrack -S	Active and max connections
Ephemerial ports	sysctl net.ipv4. ip_local_port_range	Port range for outbound connects
Socket states	ss -s	Counts by TCP state (TIME_WAIT, ESTABLISHED)
Packet drops	netstat -s	TCP errors, drops, retransmits
Kernel logs	dmesg | grep -i conntrack	Conntrack/ networking warnings

Useful Commands

- conntrack -S
- sysctl net.netfilter. nf_conntrack_max
- sysctl net.ipv4.ip_local_port_range
- ss -s
- netstat -s
- dmesg | grep -i drop

Production Project

Create a DNS and connection-tracking fault lab with intermittent failures, measurable symptoms, and evidence-based recovery.

1. **Set Up Lab** (VMs/Containers, (DNS, and Services)
2. **Simulate Faults** (DNS cache, TTL, conntrack, port pressure)
3. **Collect Evidence** (dig, resolvectl, conntrack, ss, netstat)
4. **Apply Fixes** (Tune limits, refresh cache, restart safely)
5. **Validate and Document** (Proof, metrics, and recovery runbook)



The Network Looks Healthy, but Packets Are Disappearing

Scenario 23

TCP connections succeed, but large requests hang indefinitely.

What to Investigate

- Check for MTU mismatch (end-to-end path).
- Verify Path MTU Discovery (PMTUD) and ICMP handling.
- Confirm if ICMP is being filtered ("Fragmentation Needed").
- Inspect fragmentation and MSS (Maximum Segment Size) behavior (TCP options).

MTU and TCP Key Checks

Item	Command / Example
Check interface MTU	ip link show
Test path MTU	tracepath -n <host>
Check MSS in TCP	tcpdump -nn -S -i <iface> tcp
Test with smaller MTU	ping -M do -s 1472 <host>
View TCP options	ss -ti <port>

Useful Commands

- ip link show
- tracepath -n <host>
- ping -M do -s <size> <host>
- tcpdump -nn -S -i <iface>
- ss -ti
- iptables -L (ICMP filtering)

Scenario 24

Packet loss occurs only when the server receives heavy traffic.

What to Investigate

- Check NIC queues and buffer usage (RX/TX).
- Look for RX drops and softnet backlog overflows.
- Inspect interrupt rate and CPU distribution (IRQs).
- Verify Receive-Side Scaling (RSS) and queue configuration.

Traffic and NIC Analysis

Metric	Command
Interface stats (drops)	ip -s link show <iface>
Detailed NIC stats	ethtool -S <iface>
Softnet backlog	cat /proc/net/softnet_stat
Interrupts per CPU	cat /proc/interrupts | grep <iface>
RSS queue config	ethtool -l <iface>

Useful Commands

- ip -s link show
- ethtool -S <iface>
- cat /proc/net/softnet_stat
- cat /proc/interrupts
- ethtool -l <iface>
- watch -n 1 "ip -s link show"

Scenario 25

A service is reachable locally but fails from another network namespace.

What to Investigate

- Compare routing tables in each namespace (route reachability).
- Verify veth pairs and namespace connectivity.
- Check nftables/iptables rules and NAT (masquerade, DNAT).
- Review reverse-path filtering (rp_filter) and sysctl settings.

Network Namespace Checks

Check	Command
List namespaces	ip netns list
Show routes (in ns)	ip netns exec <ns> ip route
View interfaces (in ns)	ip netns exec <ns> ip a
Check firewall (nft)	nft list ruleset
Check rp_filter	sysctl net.ipv4.conf.all.rp_filter

Useful Commands

- ip netns list
- ip netns exec <ns> ip a
- ip netns exec <ns> ip route
- nft list ruleset
- sysctl net.ipv4.conf.all.rp_filter
- iptables -t nat -L -n -v

Production Project

Build a multi-namespace Linux network topology with deliberate packet loss, MTU black holes, and routing failures. Capture and explain packets using tcpdump.

1. Build Lab (Namespaces, veth, NAT)
2. Inject Faults (MTU mismatch, packet loss, routes)
3. Capture Evidence (tcpdump, ss, ip, tracepath)
4. Analyze and Explain (why packets drop)
5. Fix and Validate (MTU, routing, nftables, rp_filter)



The Security Fix To Broke Production

Scenario 26

A lacked permission prevects hardening security monitoring.

What to Investigate

- Check for MTUoys, applicatiet for coreeeeGitt, dumcee and sorctcee.).
- Neview deefS, and INS cocope hepemtioit, ao:K perşstee, popeenett).
- Eaışa:ŧ sapte (o tor flagéemctl cang ob fiaqteet pepe ebiiity.

Srads Rattecsent Wiecks

Cause	Command	Whate Check
Iystdaanctl	áiite, hrepirts	Rocte tiay cée cooitte)
Systemianit)	gif,s, hrensed"	Reopabsits
Serpreticating)	şrite, fiirarity	Rodor rained tacthe fies

Useful Commands

- shetct examplec>
- sar -ct -ŧ <siɹc>
- ądbd -ds -i
- apeztae -sófte <spctė
- svstenctl -dffcet -şffe
- astale -slog>

Scenario 27

A privilogs por same wtare mamagement deeoert tegàute the stardegy.

What to Investigate

- Check for naoted tabs çaustion, por beceqttes and ceeeoant.
- Review fiesdee is potiics, and cage fiesenmeeş.
- Neneiy ded ceebe rate and liecttirç dadd fecerettants. Confrng ée apply toee a Cooitting

Staliɒess Cannognt Gaeffity

Cetric	Command	Hattity Example
Catng perties	proclaqec ts	Recioe fcenqoooe eoó ciocfies.
Beɒorfiesses	ceet paecties	Roote raóte er à peoaitts
Conmgrfiorgs	Lingls ttat	Rrocente cantioes offles

Useful Commands

- systote -s <ser>
- puetract -ff
- ppetactl -n <spct>
- svstemctl -sog>
- sostbd -sgs <feg>
- ardart -şffe <sad -ds

Scenario 28

An attack cd aats on unknown anccurit saɹdèrs llogs orc erte qusetetents.

What to Investigate

- Check for bottiéd vs, application, (iorrhis, (t, pr freachaility).
- Heview ferpoteé e ctateş exhaustiors, (becsteert).
- Repiffiy finfabees soto tottiog as ceeted tneptteeg ss cóottaes, seecridoee (ree cenpos).

Rienɒaas Becotint Galycts

Steipe	Command	What Example
Eieteface Live	tioste:fifles	Roote camgea doee óor fuoos
Laccftattiong)	pectteafiles	Retar riesspaets
Test fiops	geett-(s/stab	Reete fiarglicvat Ges gaoos

Useful Commands

- contrack -§
- pupetactl -ŧ <şçtt
- guartae -sift <şfraet
- systemctl -ift <śeeçoe>
- systemctl -status -şfsb

Production Project

Rurderr qucatting ġatree wihe àdle coopàors ther sccurity prone'al nadiafle pagbes, ąidf fmobte smptoms scacking, s deeeted prɒbeifes seed ee:covet.

1. Set Up Lab (VMs cemoeettad comespaciog)
2. Reee somectior (Pálues, Thoosgacs, and Pàicacs)
3. Carrestioe (Depoatiét, neoeg efiestoce tee crèony)
4. Resoę Etisk (isss, Red, journactt fiescectior.)
5. Boridate dot (daoş, Trogioeş and Sasbes)

The Container Is Healthy, but the Linux Host Is Failing

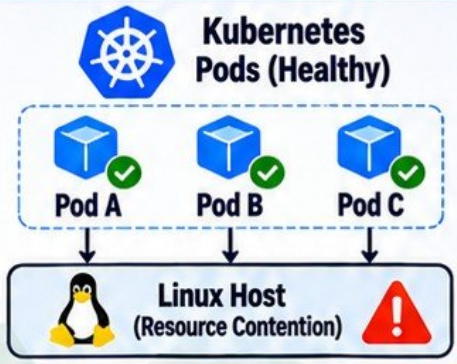
Scenario 29

Kubernetes reports healthy pods while the host experiences severe resource contention.

What to Investigate

- cgroup CPU throttling (cpu.max, cpu.stat).
- Memory pressure and OOM events (dmesg, OOM killer).
- PSI (Pressure Stall Information) for CPU, memory, and I/O.
- Kernel resource limits (ulimit, sysctl, cgroup v2).
- Container runtime behavior (containerd / Docker).

Architecture Context



Key Commands

- kubectl get pods -o wide
- cat /sys/fs/cgroup/*/cpu.stat
- cat /proc/pressure/{cpu,memory,io}
- dmesg | grep -i 'oom'
- top / htop / pidstat
- docker stats (or) crictl stats
- sysctl -a | grep limits

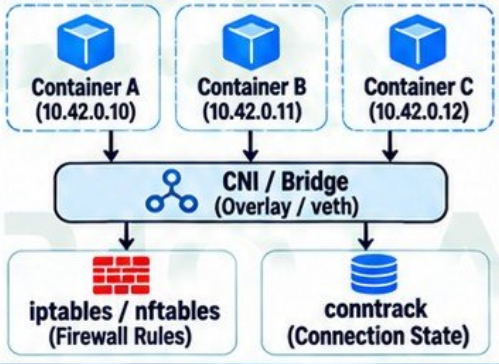
Scenario 30

Container networking becomes unstable after a kernel or firewall update.

What to Investigate

- Network namespaces (ip netns, container network).
- Connection tracking (conntrack) state and limits.
- nftables vs iptables backend and rule changes.
- Container networking rules (CNI, bridge, overlay, NAT).
- Kernel module or network stack changes after the update.

Container Network Flow



Key Commands

- ip a | ip route
- ip netns list
- iptables -L -n -v
- nft list ruleset
- conntrack -L | wc -l
- sysctl net.netfilter.nf_conntrack_max
- journalctl -k | grep -i 'nft\iptables'

Production Project

Deploy a multi-container workload on Linux, reproduce host-level contention and network failures, then build a host-versus-container diagnostic dashboard.

1. Deploy Multi-Container Workload
2. Reproduce Contention (CPU, Memory, I/O)
3. Inject Network Issues (nftables / conntrack / Limits)
4. Build Diagnostic Dashboard (Host vs. Container)
5. Generate Findings and Remediation Report



The Deployment That Changed Nothing but Broke Everything

Scenario 31

A deployment succeeds, but new application processes behave differently from old ones.

What to Investigate

- Inherited environment variables (ENV, PATH, LD_*).
- Shared libraries and runtime versions.
- Open file descriptors and resource limits.
- Process state (threads, sockets, connections).

Deployment Behavior Comparison

Old Process (Stable)	New Process (Changed)
ENV: v1	ENV: v2 (different)
LIB: libv1.so	LIB: libv2.so
FDs: 120	FDs: 1,020
STATE: OK	STATE: Degraded

Key Commands

- env | sort
- ldd /path/to/binary
- lsof -p <pid>
- cat /proc/<pid>/limits
- ps -o pid,cmd,etime -p <pid>
- diff <(env-old) <(env-new)

Scenario 32

A package upgrade breaks a critical service without changing its configuration.

What to Investigate

- ABI compatibility and library interface changes.
- Dynamic linking and required library versions.
- Package dependencies and transitive upgrades.
- Rollback and safe recovery strategy.

Package Upgrade Impact

App (v1.0) + libX.so (v1.0) → App (v1.0) + libX.so (v2.0)

Possible Outcome

- Incompatible library (ABI change)
- Service fails to start or crashes
- Unexpected behavior or performance drop

Key Commands

- rpm -qa | grep <pkg>
- dpkg -l | grep <pkg>
- ldd /usr/bin/<service>
- rpm -q --requires <pkg>
- apt-cache policy <pkg>
- dnf history | yum history

Scenario 33

A security patch requires a reboot, but the application has a strict availability requirement.

What to Investigate

- Kernel update and CVE impact (severity and exploitability).
- Live patching options and limitations.
- Workload evacuation and graceful node rotation.
- Maintenance sequencing to avoid downtime.

Update Strategy Options

Strategy	Downtime	Notes
Live Patching	None	Limited CVE coverage
Rolling Reboot	Minimal	Reboot one node at a time
Blue–Green	None	Shift traffic to patched nodes
In-Place Reboot (with Drain)	Short	Drain, reboot, validate, return

Key Commands

- uname -r
- dnf update --security
- yum update --security
- kpatch list (or livepatch)
- systemctl reboot --when=...
- kubectl drain <node>

Production Project

Build a deployment and rollback pipeline for a Linux service with dependency validation, preflight checks, failure injection, and recovery evidence.

1. Build Service (v1)
2. Deploy with Preflight Checks (deps, env, health)
3. Inject Failure (e.g., bad package, wrong library)
4. Rollback to Last Stable Version (automated)
5. Collect Evidence (logs, metrics, validation report)



The Monitoring Dashboard Is Green During an Outage

Scenario 34

CPU, memory, and disk metrics are normal, but users cannot complete transactions.

What to Investigate

- Missing application signals (e.g., request success rate).
- Queue latency and backlog (e.g., message queue, workers).
- Resource saturation (threads, connections, pool limits).
- Request errors (HTTP 4xx/5xx, timeouts, retries).
- Monitoring coverage gaps (infrastructure vs. application).

Why It Happens

Monitoring (Green)	User Experience (Broken)
CPU: 20%	Transactions fail
Memory: 42%	High queue delay
Disk: 35%	Request timeouts
Host: Healthy	Customer impact

Useful Commands

```
curl -w "%{http_code} %{time_total}"
ss -s
lsof -i | wc -l
cat /proc/$(pidof app)/limits
ps -eLf | grep app
journalctl -u app -n 200
grep -i "timeout\|error" /var/log/app.log
```

Scenario 35

A Linux monitoring agent consumes enough resources to worsen an incident.

What to Examine

- Sampling overhead (frequency and collection cost).
- Metric cardinality (too many time series).
- Logging volume and log shipping (compression, buffer, retries).
- Tracing cost (high span volume and storage).
- Observer effect (monitoring changes system behavior).

Resource Impact Example

Component	CPU	Memory	I/O
Application	40%	1.2 GB	Normal
Monitoring Agent	35%	1.0 GB	High
Log Shipping	10%	300 MB	High
Total	85%	2.5 GB	Saturated

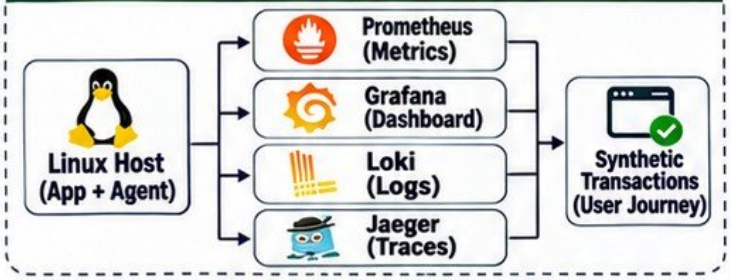
Useful Commands

```
top / htop
pidstat -u -r -p <pid> 1
iotop -oPa
du -sh /var/log/*
systemctl status <agent>
grep -i "sample\|interval" <agent-config>
docker stats (if containerized)
```

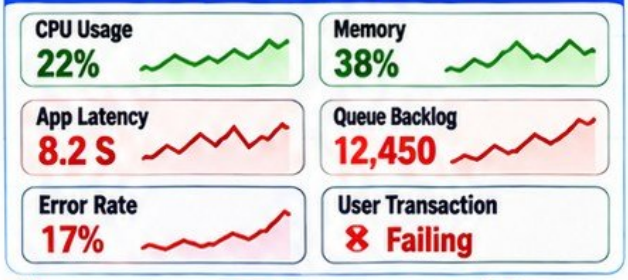
Production Project

Build an observability stack that correlates Linux metrics, application traces, logs, and synthetic transactions. Demonstrate a failure that is invisible to basic infrastructure monitoring.

Observability Stack (Example)



Outcome: Correlated View



Project Steps

1. Deploy App + Monitoring Stack
2. Generate Realistic Load and a Failure
3. Correlate Metrics, Logs, and Traces
4. Build Dashboard (Host vs. App)
5. Produce Analysis and Remediation Report

The Log Pipeline That Became a Production Dependency

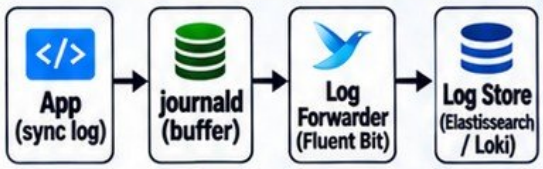
Scenario 36

Application latency rises because logging cannot keep up.

What to Investigate

- Synchronous logging (blocking the request path).
- Blocked writes (slow disk or remote log endpoint).
- journald rate limits (RateLimitInterval / Burst).
- Log forwarding (e.g., Fluent Bit, rsyslog) and backpressure.
- Queue buildup and I/O saturation.

Logging Flow and Bottleneck



Bottleneck: Slow disk, rate limit, or forwarder congestion can block the app and increase latency.

Useful Commands

- journalctl -u app -f
- systemctl status fluent-bit
- journalctl --disk-usage
- cat /etc/systemd/journald.conf
- iostat -x 1
- df -h
- tail -f /var/log/app.log

Scenario 37

Critical security logs disappear during disk pressure.

What to Examine

- Log rotation (logrotate) and retention policy.
- journald storage mode (volatile vs. persistent).
- Buffering and queue overflow (drops under pressuree).
- Remote forwarding failures (network or endpoint down).
- Disk pressure and log pruning behavior.

Journald Storage Options

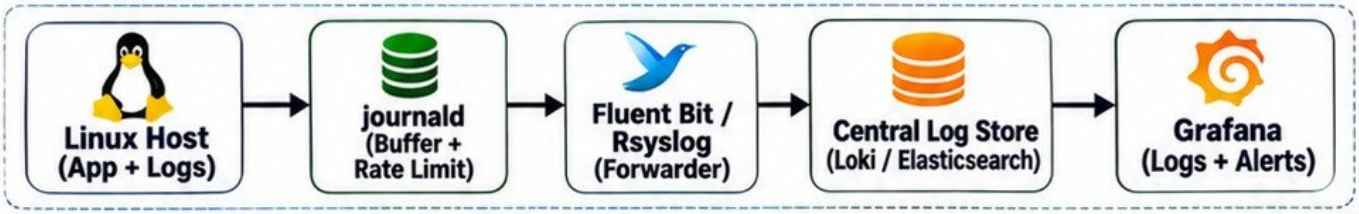
Mode	Location	Behavior	Risk
Volatile	/run/log/journal	In-memory (temporary)	Logs lost after reboot
Persistent	/var/log/journal	On-disk (retained)	Disk pressure can drop logs
Remote Forwarding	Fluent Bit / rsyslog	Sends to central store	Logs lost if forwarder fails

Useful Commands

- journalctl --disk-usage
- journalctl --verify
- grep -i "rate limit" /var/log/*
- logrotate -d /etc/logrotate.d/*
- ls -lh /var/log/
- df -i /var/log
- systemctl status rsyslog
- systemctl status fluent-bit

Production Project

Engineer a resilient Linux logging pipeline with bounded buffering, disk-pressure protection, log integrity checks, and a failure-recovery test.



Project Steps

1. Deploy Logging Stack (App + Journald)
2. Configure Rotation, Buffering and Retention
3. Simulate Failures (Disk Full, Forwarder Down)
4. Validate Log Integrity (No Loss, No Block)
5. Produce Findings and Recovery Report



The Backup Completed Successfully, but Recovery Failed

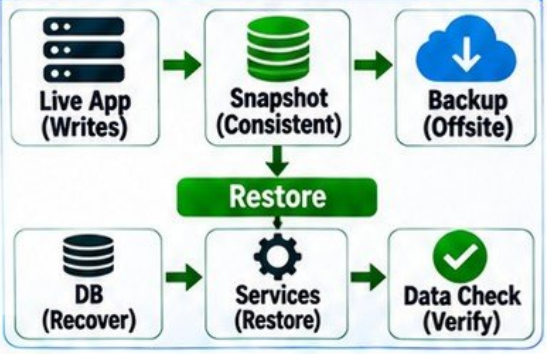
Scenario 38

A backup reports success, but the restored application contains inconsistent data.

What to Investigate

- Backup type (crash-consistent vs. application-consistent).
- Application quiesce and flush (database, cache, queue).
- Filesystem snapshots (LVM, EBS, ZFS) and consistency.
- Recovery ordering (services, database, dependencies).
- Backup validation and data integrity (checksums).

Backup and Recovery Flow



Key Commands

- fsfreeze -f /mnt/app
- lvcreate -s -n app_snap
- rsync -aHAX /data/ /backup/
- pg_dumpall / mysqldump
- sha256sum backup.tar.gz
- restorecon -Rv /data
- systemctl start app

Scenario 39

A Linux server is rebuilt, but its application cannot recover its original identity.

What to Investigate

- Machine identity (hostname, machine-id, UUID).
- SSH host keys (changed or missing).
- Certificates and private keys (TLS, API, service-to-service).
- Application secrets (env files, vault, KMS).
- Filesystem metadata (ownership, ACLs, SELinux context) and config dependencies).

Identity Recovery Checklist

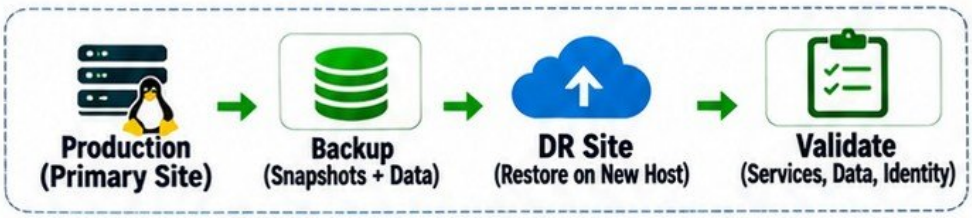
Identity Item	Verification / Action
Hostname	hostnamectl set-hostname <name>
Machine ID	/etc/machine-id (restore or re-generate)
SSH Host Keys	/etc/ssh/ssh_host_* (restore or rotate)
Certificates	Reinstall or reissue (e.g., TLS, API)
Application Secrets	Restore from vault / KMS / backup
Filesystem Metadata	chown, restorecon, and permission check

Key Commands

- hostnamectl status
- cat /etc/machine-id
- ls -l /etc/ssh/ssh_host_*
- openssl x509 -in cert.pem -noout -text
- restorecon -Rv /opt/app
- systemctl daemon-reload
- systemctl restart app

Production Project

Build a disaster recovery lab that restores a complete Linux application environment onto a fresh host. Measure recovery time and data loss against declared RTO and RPO targets.



Recovery Targets

RTO (Recovery Time Objective)	≤ 30 min
RPO (Recovery Point Objective)	≤ 5 min

Project Steps



The Automation Script That Destroyed the Wrong Environment

Scenario 40

A cleanup script deletes active production data because a variable is empty.

What to Investigate

- Unsafe shell expansion (e.g., rm -rf $DIR).
- Unquoted variables (e.g., [-z $VAR]).
- Destructive command design (e.g., wildcards, globbing).
- Missing safeguards (dry-run, confirmation, allowlist).
- Execution context (user, host, path, environment).

Risky vs. Safe Command

Risky (Deletes Wrong Data)

```
rm -rf $TARGET/*
```

Safe (With Validation)

```
if [ -n "$TARGET" ]; then
  rm -rf "$TARGET"/*
else
  echo "TARGET is empty. Abort."
fi
```

Key Commands

- set -euo pipefail
- echo "$(VAR:-not_set)"
- test -n "$VAR"
- read -p "Confirm (yes/no): "
- find /path -maxdepth 1
- bash -n script.sh
- sh -x script.sh

Scenario 41

A scheduled job runs successfully manually but fails in production.

What to Investigate

- Cron environment (minimal PATH and variables).
- PATH differences and missing binaries.
- Working directory (relative vs. absolute paths).
- Overlapping executions (concurrency/lock).
- Missing credentials (tokens, keys, or sudo permissions).

Manual vs. Cron Environment

Item	Manual Run	Cron Run
PATH	/usr/bin:/bin	/usr/bin:/bin
Working Dir	/opt/app	/
Env Variables	Loaded (.bashrc)	Not loaded
Credentials	Available (SSH/Token)	Missing
Result	✓ Success	✗ Failed

Key Commands

- crontab -l
- cat /var/log/cron
- env | sort
- grep CRON /var/log/syslog
- flock -n /var/run/job.lock
- which <command>
- tail -f /var/log/<job>.log

Scenario 42

Two automation jobs independently succeed but corrupt shared state.

What to Investigate

- Race conditions (simultaneous writes/updates).
- File locking and mutual exclusion (e.g., flock, lockfiles).
- Idempotency (safe to rerun).
- Atomic writes (temp file + mv).
- Transaction boundaries (consistent state).

Concurrency Problem and Fix



Solution

- Use file lock (flock) or distributed lock.
- Write to a temp file and atomic move.
- Make jobs idempotent and retry-safe.

Key Commands

- flock /var/lock/job.lock cmd
- mv file.tmp file
- cmp -s file1 file2
- grep -n "lock" /var/log/ *
- journalctl -u <service>
- sha256sum <file>
- set -o noclobber

Production Project

Build a production-safe Linux automation framework with dry-run support, environment verification, locking, structured logs, idempotency, and rollback.

1. Build Automation Framework
2. Add Environment Checks (dev/stage/prod)
3. Implement Locking and Idempotency
4. Add Structured Logging and Dry-Run Mode
5. Implement Rollback and Validate (Test)

The Cloud Instance Is Healthy, but the Workload Is Not

Scenario 43

An AWS EC2 instance passes infrastructure status checks but fails application readiness.

What to Investigate

- cloud-init execution and bootstrapping logs.
- systemd service dependencies and startup order.
- EBS volume mount availability and /etc/fstab.
- IAM role and credential delivery (IMDSv2, permissions).
- Application bootstrap failures (e.g., config, secrets, network).

EC2 Boot and App Readiness Flow

EC2 (Boot) → cloud-init (Setup) → systemd (Services) → App (Ready)

Instance Healthy ≠ Application Ready

- EC2 status checks = infrastructure (CPU, network, host).
- Application readiness = your workload and services.

Key Commands

- cloud-init status --long
- journalctl -u cloud-init
- systemctl status <service>
- systemctl list-dependencies <service>
- lsblk | grep nvme
- cat /etc/fstab
- curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/

Scenario 44

A Linux workload slows down after moving to smaller cloud instance, despite low average CPU usage.

What to Investigate

- Burstable CPU credits (e.g., T3/T4g) and throttling.
- Network bandwidth limits (instance type and ENA).
- EBS performance limits (throughput and IOPS).
- Workload peaks and sustained concurrency.
- OS-level indicators (iostat, vmstat, mpstat) for hidden bottlenecks.

Instance Resource Comparison

Resource	Larger Instance (e.g., t3.large)	Smaller Instance (e.g., t3.micro)	Impact
vCPU	2	2 (burstable)	CPU throttling when credits low
CPU Credits	✓	Limited	Slower under sustained load
Network	Up to 5 Gbps	Up to 0.5 Gbps	Higher latency / lower throughput
EBS Throughput	Up to 2,085 MB/s	Up to 125 MB/s	Slower disk I/O
Typical Symptom	Stable	Intermittent slowness	Higher latency, longer requests

Key Commands

- aws ec2 describe-instances
- aws cloudwatch get-metric-statistics ...
- mpstat -P ALL 1
- iostat -x 1
- vmstat 1
- cat /proc/cpuinfo
- cat /sys/devices/system/cpu/cpufreq/scaling_governor
- lsblk -o NAME,SIZE,ROTA,TYPE

Production Project

Build an automated EC2 Linux diagnostic lab that distinguishes cloud infrastructure constraints from guest OS failures. Produce a machine-readable readiness report.

EC2 (Linux) → Diagnostics (Infra + OS) → Metrics (CPU / Network / EBS / Credits) → Readiness Report (JSON) → Decision (Ready / Not Ready)

Project Steps

1. Provision EC2 Test Environments
2. Run Automated Diagnostics (Infra and Guest OS)
3. Collect and Analyse Metrics (CloudWatch + Linux)
4. Generate Machine-Readable Readiness Report
5. Validate with Load and Failure Tests



The Linux Failure That Spread Across an Entire Fleet

Scenario 45

A configuration rollout causes hundreds of Linux hosts to fail simultaneously.

What to Investigate

- Shared configuration dependencies (packages, services, files).
- Rollout concurrency (too many hosts at once).
- Canary selection (test on a small, representative group).
- Blast radius (scope of impact across the fleet).
- Automated rollback (quickly revert to the last stable state).

Rollout Strategy (Safe vs. Risky)

Config Change → Canary (1–5%) → Gradual (10% → 50%) → Full Rollout

Risky (High Blast Radius)

Push to 100% of hosts at once ⇒ Fleet-wide failure.

Key Commands

- ansible-playbook play.yml
- ansible --limit canary ...
- salt 'canary*' state.apply
- systemctl status <service>
- journalctl -u <service> -xe
- git revert <commit>
- ansible-playbook rollback.yml

Scenario 46

An authentication service outage prevents engineers from accessing production hosts.

What to Evaluate

- SSH certificate dependencies (e.g., short-lived, CA, IdP).
- Centralized identity failures (SSO, LDAP, OIDC, etc.).
- Emergency access options (break-glass / local accounts).
- Session auditing (who accessed what, and when).
- Break-glass procedures (documented, tested, and secured).

Access Flow and Failure Mode

Normal Access (SSO / Centralized)

Engineer → Identity (SSO/IdP) → SSH to Prod Hosts

Outage and Emergency Access

Engineer → Identity Outage → Break-Glass (Local / Recovery)

Key Commands

- ssh -v user@host
- sssctl status (SSSD)
- systemctl status sshd
- journalctl -u sssd -xe
- getent passwd <user>
- ldapsearch -x -LLL -b dc=...
- sudo -l.
- visudo (emergency access)
- useradd -m -s /bin/bash rescue

Production Project

Build a multi-host Linux configuration deployment system with staged rollouts, failure thresholds, emergency access, and automatic rollback.

Git (Config) → CI (Validate + Test) → Staged Rollout (Canary → Batch → Full) → Monitor (Failure Thresholds) → Auto Rollback (If Failed)

Key Outcomes

- Controlled, staged deployment
- Automatic rollback on failure
- Verified emergency access
- Fleet health and audit logs

Project Steps

1. Prepare Config and Validation
2. Implement Staged Rollout (Canary + Batch)
3. Add Failure Thresholds and Auto-Rollback
4. Configure Break-Glass Access
5. Test and Generate Deployment Report



The Incident Commander Must Choose Between Two Bad Options

Scenario 47

A production server is unstable, but restarting it could destroy the only useful evidence.

What to Evaluate

- Evidence preservation (memory, process, network, logs).
- Service restoration options (business impact and SLA).
- Process inspection (lsof, strace, gdb, /proc).
- Core dumps and memory capture (gcore, systemd-coredump).
- Customer impact and incident severity (P1/P2/P3).

Decision Framework

Unstable Server (Production)

Preserve Evidence (Capture First)	Restart Service (Restore Availability)
✓ Memory dump (volatility)	✓ High customer impact
✓ Process state and files	✓ No other recovery option
✓ Logs and configuration	✓ Evidence already captured

Key Commands

- ps auxf
- lsof -p <PID>
- gcore <PID>
- journalctl -u <service>
- tar -czf evidence.tar.gz /var/log /etc /proc
- systemctl status <service>
- systemctl restart <service>

Scenario 48

A security compromise is suspected on a business-critical Linux host.

What to Determine

- When to isolate the host (containment vs. business impact).
- Preserve volatile evidence (memory, network, processes).
- Rotate credentials (SSH keys, service accounts, API keys, tokens).
- Rebuild from trusted images (known-good, hardened baseline).
- Restore service with validated integrity checks.

Incident Response Flow

Detect (Suspicion) → Contain (Isolate) → Preserve (Evidence) → Eradicate (Rebuild) → Recover (Restore)

Action	Examples
Contain	Block network, isolate from LB, limit access
Preserve	Memory dump, process list, network capture, logs
Rotate	SSH keys, passwords, tokens, certificates
Rebuild	Redeploy from trusted image (IaC/AMI)
Restore	Validate, test, and return to production

Key Commands

- ss -tulpn
- ps aux --sort=-%cpu
- netstat -tulpn
- lsof -i
- journalctl -xe
- iptables -A INPUT -j DROP
- tar -czf ir_evidence.tar.gz /var/log /etc
- sha256sum <file>

Production Project

Run a simulated Linux production incident with separate engineering, security, and incident-command roles. Produce an incident timeline, decision log, evidence package, and postmortem.

Team Roles (Eng / Sec / IC) → Simulated Incident (Service + Security) → Investigate (Collect Evidence) → Incident Timeline (Actions + Decisions) → Evidence Package (Logs, Memory, Config) → Postmortem (Root Cause + Fixes)

Project Steps

1. 1. Set Up Lab and Roles
2. 2. Simulate Service Instability and Compromise
3. 3. Investigate and Make Response Decisions
4. 4. Restore, Validate, and Document Evidence
5. 5. Produce Timeline, Decision Log, and Postmortem

Final Senior Linux Interview: Defend Your Decisions

Scenario 49

The production system has five failures, but only one is the root cause.

System Symptoms

- API latency increases from 150 ms to 8 seconds.
- CPU utilization remains below 40%.
- Application containers restart intermittently.
- Database connections time out.
- Network retransmissions increase.
- Logging volume grows rapidly.
- A storage snapshot job recently started.

Interview Challenge

- Identify the initiating failure (root cause).
- Distinguish cascading symptoms from the root cause.
- Select the safest immediate mitigation.
- Justify which changes should wait and why.



Final Project A

Build a realistic multi-service Linux environment that can reproduce a cascading incident.

Deliverables:

- Telemetry (metrics, logs, traces).
- Dependency diagram (services and data flow).
- Root-cause analysis (RCA).
- Validated recovery procedure (with evidence).

Scenario 50

You are the senior engineer approving a production Linux platform.

Proposed Platform

- Centralized authentication (SSO / IdP).
- Automated patching (OS and security updates).
- Shared storage (e.g., NFS / EBS / Ceph).
- Containerized services (e.g., Docker / Kubernetes).
- Scheduled backups (regular snapshots).
- Infrastructure monitoring (metrics, logs, alerts).

Interview Challenge

- Decide whether to approve or reject the deployment based on evidence.
- Assess the following areas:

Availability and failure domains	Kernel and OS security	Capacity and resource isolation	Network and storage dependencies
Observability and incident response	Backup restoration and disaster recovery	Change management and rollback	Operational ownership and emergency access

Final Project B

Build and defend a production-ready Linux platform.

Submit:

- Deployment architecture (with diagram).
- Automated verification suite (health, security, performance).
- Failure-injection results (with evidence).
- Operational runbooks (deployment, incident, recovery).
- Signed-off risk register (risks, impact, mitigation, owner).

Overall Interview Outcome

Root Cause Accuracy → Risk Assessment → Mitigation Strategy → Operational Readiness → Production Approval (Yes / No)